LAB REPORT 03
Continuous and Discrete-Time System Responses

Exercise 01: System Response to a Continuous-Time Signal

> Suppose that a system is described by the impulse response $h(t) = \sin(3\pi t)\,[u(t) - u(t-5)]$. Compute and plot the system response to the input signal $x(t) = \cos(2\pi t)\,[u(t) - u(t-5)]$.

MATLAB Implementation Code:

```
% 1. Define the continuous time approximation
dt = 0.001;          % Very small time step
t = -2:dt:8;         % Time vector covering before and after the signals

% ---> ADDED: Clear command window and print setup information
clc;
fprintf('--- Continuous-Time Convolution Analysis ---\n');
fprintf('Time step (dt) set to: %.3f\n', dt);

% 2. Define the window: [u(t) - u(t-5)]
% This creates an array of 1s between 0 and 5, and 0s elsewhere
window = (t >= 0) & (t <= 5);

% 3. Generate x(t) and h(t)
x = cos(2*pi*t) .* window;
h = sin(3*pi*t) .* window;

% ---> ADDED: Print signal status
fprintf('Signals x(t) and h(t) generated successfully for t = [0, 5].\n');
fprintf('Calculating continuous convolution approximation...\n');

% 4. Compute the continuous convolution approximation
y = conv(x, h) * dt;

% 5. Determine the output time vector
% The output time starts at the sum of the input start times
t_start = t(1) + t(1);
ty = t_start : dt : t_start + (length(y)-1)*dt;

% ---> ADDED: Print useful analytical outputs from the convolution
fprintf('Convolution complete.\n');
fprintf('-------------------------------------------------\n');
fprintf('Output Time Range          : [%.2f, %.2f] seconds\n', ty(1), ty(end));
fprintf('Maximum Output Amplitude   : %.4f\n', max(y));
fprintf('Minimum Output Amplitude   : %.4f\n', min(y));
fprintf('Output Vector Length       : %d samples\n', length(y));
fprintf('-------------------------------------------------\n');
fprintf('Generating plot...\n\n');
```

```
% 6. Visualize the continuous signals
figure('Name', 'Continuous-Time Convolution');
subplot(3, 1, 1);
plot(t, x, 'LineWidth', 1.5);
title('Input Signal x(t)');
xlabel('Time t'); ylabel('Amplitude');
grid on;

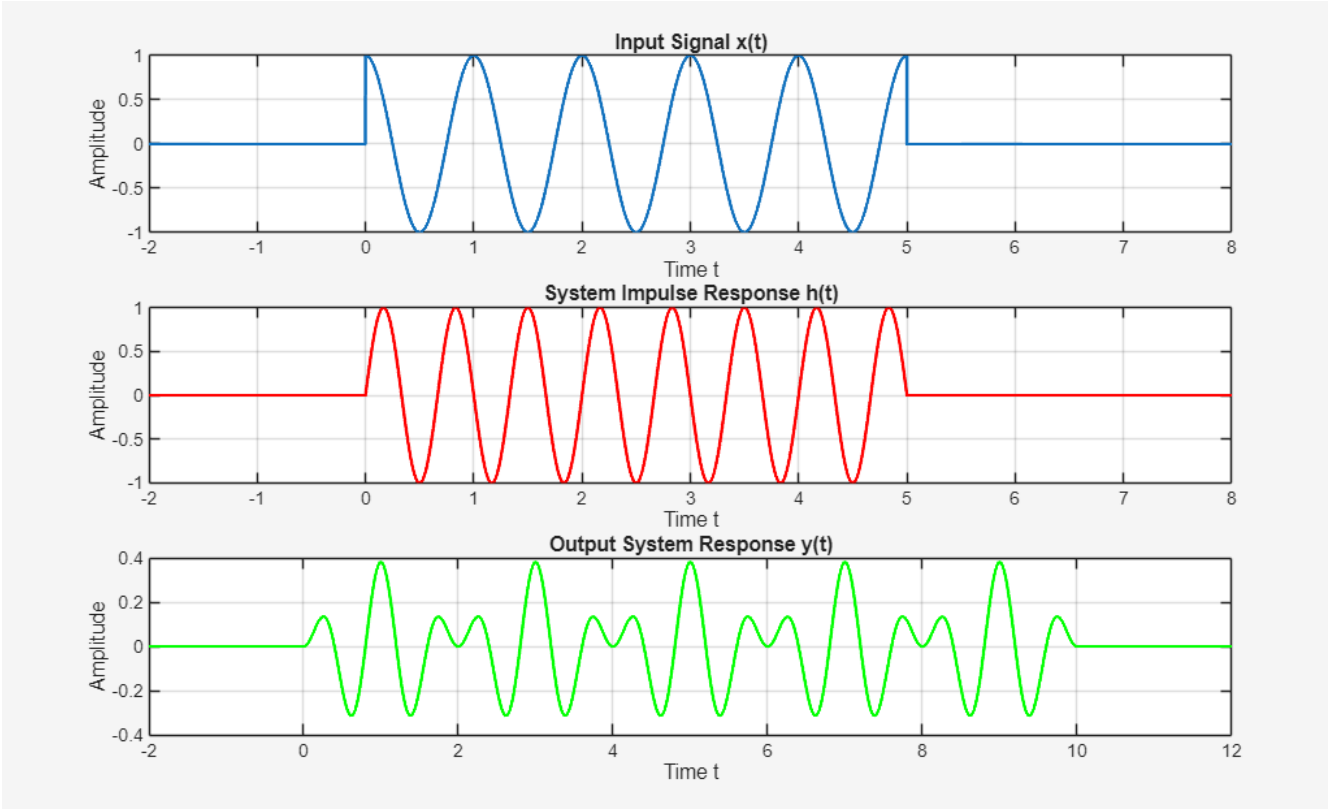
subplot(3, 1, 2);
plot(t, h, 'LineWidth', 1.5, 'Color', 'r');
title('System Impulse Response h(t)');
xlabel('Time t'); ylabel('Amplitude');
grid on;

subplot(3, 1, 3);
plot(ty, y, 'LineWidth', 1.5, 'Color', 'g');
title('Output System Response y(t)');
xlabel('Time t'); ylabel('Amplitude');
xlim([-2 12]); % Focus the view on the relevant time frame
grid on;
```

Simulation Output Plot & Command Window:

Command Window

```
--- Continuous-Time Convolution Analysis ---
Time step (dt) set to: 0.001
Signals x(t) and h(t) generated successfully for t = [0, 5].
Calculating continuous convolution approximation...
Convolution complete.
--------------------------------------------------
Output Time Range          : [-4.00, 16.00] seconds
Maximum Output Amplitude   : 0.3820
Minimum Output Amplitude   : -0.3123
Output Vector Length       : 20001 samples
--------------------------------------------------
Generating plot...
```



Exercise 02: Continuous-Time Convolution of Exponential and Trigonometric Signals

> Compute and plot the convolution between the signals x(t) = cos(πt)[u(t+2) - u(t-3)] and h(t) = 5e^{-t}u(t).

MATLAB Implementation Code:

```
clc;
clear;
close all;

fprintf('--- Continuous-Time Convolution Analysis ---\n\n');

% 1. Define the continuous time approximation
dt = 0.001;          % Very small time step
t = -2:dt:8;         % Time vector covering before and after the signals
fprintf('Time step (dt) set to: %.3f seconds\n', dt);

% 2. Define the window: [u(t) - u(t-5)]
% This creates an array of 1s between 0 and 5, and 0s elsewhere
window = (t >= 0) & (t <= 5);

% 3. Generate x(t) and h(t)
x = cos(2*pi*t) .* window;
h = sin(3*pi*t) .* window;
```

```
fprintf('Signals x(t) and h(t) generated successfully for t = [0, 5].\n');
fprintf('Calculating continuous convolution approximation...\n');

% 4. Compute the continuous convolution approximation
y = conv(x, h) * dt;

% 5. Determine the output time vector
% The output time starts at the sum of the input start times
t_start = t(1) + t(1);
ty = t_start : dt : t_start + (length(y)-1)*dt;

% ---> ADDED: Print useful analytical outputs from the convolution to the Command Window
fprintf('Convolution complete.\n');
fprintf('--------------------------------------------------\n');
fprintf('Output Time Range          : [%.2f, %.2f] seconds\n', ty(1), ty(end));
fprintf('Maximum Output Amplitude   : %.4f\n', max(y));
fprintf('Minimum Output Amplitude   : %.4f\n', min(y));
fprintf('Output Vector Length       : %d samples\n', length(y));
fprintf('--------------------------------------------------\n');
fprintf('Generating plot...\n\n');

% 6. Visualize the continuous signals
% ---> MODIFIED: Added 'Color', 'w' for a clean white background for lab reports
figure('Color', 'w', 'Name', 'Continuous-Time Convolution');

subplot(3, 1, 1);
plot(t, x, 'LineWidth', 1.5);
title('Input Signal x(t)');
xlabel('Time t'); ylabel('Amplitude');
grid on;

subplot(3, 1, 2);
plot(t, h, 'LineWidth', 1.5, 'Color', 'r');
title('System Impulse Response h(t)');
xlabel('Time t'); ylabel('Amplitude');
grid on;

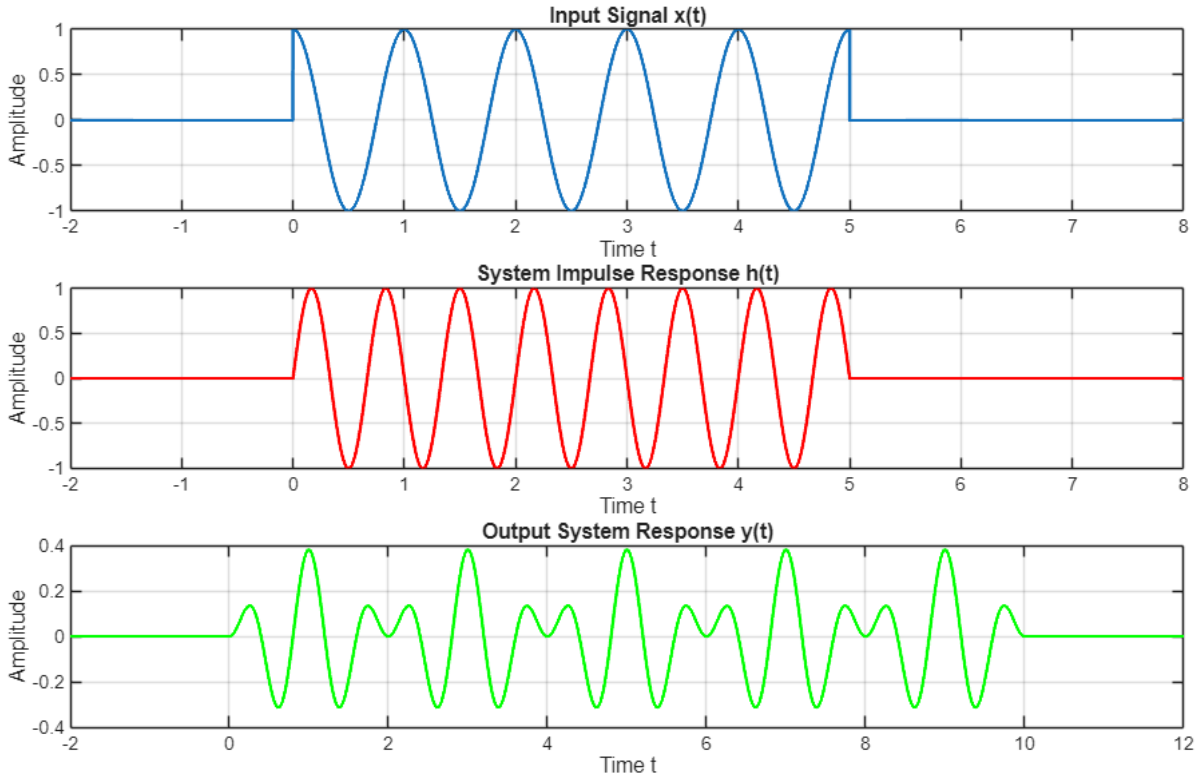
subplot(3, 1, 3);
plot(ty, y, 'LineWidth', 1.5, 'Color', 'g');
title('Output System Response y(t)');
xlabel('Time t'); ylabel('Amplitude');
xlim([-2 12]); % Focus the view on the relevant time frame
grid on;
```

Simulation Output Plot & Command Window:

Command Window

```
--- Continuous-Time Convolution Analysis ---

Time step (dt) set to: 0.001 seconds
Signals x(t) and h(t) generated successfully for t = [0, 5].
Calculating continuous convolution approximation...
Convolution complete.
--------------------------------------------------
Output Time Range          : [-4.00, 16.00] seconds
Maximum Output Amplitude   : 0.3820
Minimum Output Amplitude   : -0.3123
Output Vector Length       : 20001 samples
--------------------------------------------------
Generating plot...
```



Exercise 03: Discrete-Time Difference Equation Responses

> Consider the system described by the difference equation y[n] + 0.1y[n-1] - y[n-2] = x[n] + 2x[n-1] and assume zero initial conditions. Compute and plot for 0 ≤ n ≤ 30:
> a. The impulse response h[n] of the system.
> b. The step response r[n] of the system.
> c. Finally, compute and plot the response y[n] of the system to the input signal: x[n] = 0.8^n cos(n), 0 ≤ n ≤ 50.

MATLAB Implementation Code:

```
clc; clear; close all;

fprintf('--- Discrete-Time System Response Analysis ---\n\n');

% 1. Extract System Coefficients
% For y[n] + 0.1y[n-1] - y[n-2] = x[n] + 2x[n-1]
b = [1, 2];          % Numerator (x coefficients)
a = [1, 0.1, -1];    % Denominator (y coefficients)

% ---> ADDED: Print extracted system properties
fprintf('System Coefficients Extracted:\n');
fprintf('Numerator (b)   : [%s]\n', num2str(b));
fprintf('Denominator (a) : [%s]\n\n', num2str(a));

% --- Part A & B: Impulse and Step Responses ---
N1 = 0:30; % Time index 0 to 30

% Generate Inputs
delta = [1, zeros(1, length(N1)-1)]; % Impulse input
u = ones(1, length(N1));             % Step input

fprintf('Calculating Impulse and Step Responses (n = %d to %d)...\n', N1(1), N1(end));

% Calculate Outputs using filter
h = filter(b, a, delta); % Part A: Impulse response
r = filter(b, a, u);     % Part B: Step response

% ---> ADDED: Print analytical outputs for Impulse and Step responses
fprintf('Impulse Response (h[n]) - Max: %.4f, Min: %.4f\n', max(h), min(h));
fprintf('Step Response (r[n])    - Max: %.4f, Min: %.4f\n\n', max(r), min(r));

% --- Part C: Response to custom signal ---
N2 = 0:50; % Time index 0 to 50

% Generate Input (Element-wise operations are required)
x = (0.8 .^ N2) .* cos(N2); % Custom input x[n]

fprintf('Calculating Custom Signal Response (n = %d to %d)...\n', N2(1), N2(end));

% Calculate Output
```

```
y = filter(b, a, x); % Part C output

% ---> ADDED: Print analytical outputs for Custom Response
fprintf('Custom Response (y[n])  - Max: %.4f, Min: %.4f\n', max(y), min(y));
fprintf('-------------------------------------------------\n');
fprintf('Generating plots...\n\n');

% --- Plotting Results ---
% ---> MODIFIED: Added a descriptive window name
figure('Color', 'w', 'Name', 'Discrete-Time System Responses');

% Plot A: Impulse Response
subplot(3, 1, 1);
stem(N1, h, 'filled'); grid on;
title('Impulse Response h[n]'); xlabel('n'); ylabel('Amplitude');

% Plot B: Step Response
subplot(3, 1, 2);
stem(N1, r, 'filled'); grid on;
title('Step Response r[n]'); xlabel('n'); ylabel('Amplitude');

% Plot C: Custom Response
subplot(3, 1, 3);
stem(N2, y, 'filled'); grid on;
title('Output y[n] for x[n] = 0.8^n cos(n)'); xlabel('n'); ylabel('Amplitude');
```

Simulation Output Plot & Command Window:

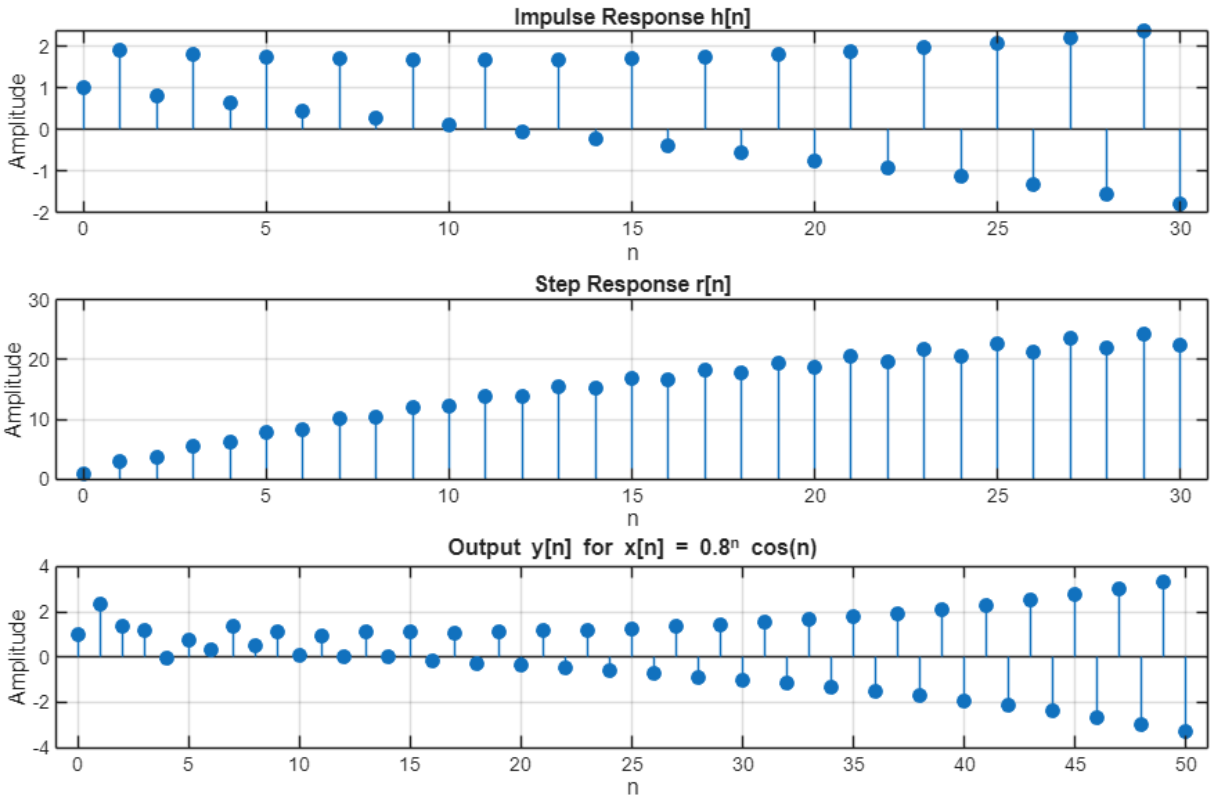
Command Window

```
--- Discrete-Time System Response Analysis ---

System Coefficients Extracted:
Numerator (b)   : [1  2]
Denominator (a) : [1         0.1          -1]

Calculating Impulse and Step Responses (n = 0 to 30)...
Impulse Response (h[n]) - Max: 2.3645, Min: -1.7930
Step Response (r[n])    - Max: 24.2848, Min: 1.0000

Calculating Custom Signal Response (n = 0 to 50)...
Custom Response (y[n])  - Max: 3.3100, Min: -3.3054
-------------------------------------------------
Generating plots...
```


Impulse Response h[n]
Amplitude
n
Step Response r[n]
Amplitude
n
Output y[n] for x[n] = 0.8^n cos(n)
Amplitude
n